



Department of Computer Science & Engineering

Course Title: Blockchain & Distributed Security Lab

Course Code: CSE 433

P2P Network

A Peer-to-Peer Chat and File-Sharing Application

GitHub Repository:

P2P-network

Submitted by:

Student Name: Tanisha Taranoon Hridy

Registration ID: 22201169

Section: D1

Dept. of Computer Science & Engineering

Submitted to:

Al-Amin Sany

Lecturer

Department of Computer Science & Engineering

University of Asia Pacific

Date of Submission: 4th October 2026

Contents

1	Abstract	3
2	Introduction	3
3	Objectives	3
4	System Architecture	4
4.1	High-Level Data Flow	4
4.2	Threading Model	5
4.3	Handshake Sequence	5
5	Application-Level Protocol (protocol.py)	5
5.1	Constants	6
5.2	Key Functions	6
5.2.1	generate_peer_id()	6
5.2.2	encode_message(message: dict) -> bytes	6
5.2.3	recv_exact(sock, num_bytes) -> bytes None	6
5.2.4	recv_message(sock) -> dict None	6
5.2.5	send_message(sock, message)	6
5.3	Message Types	7
6	Core Networking Layer (p2p_node.py)	7
6.1	State Held by a PeerNode	7
6.2	Starting and Stopping – start() / stop()	7
6.3	Accepting Connections – _accept_loop() / _handle_incoming_connection()	8
6.4	Connecting Out – connect_to_peer() / _connect_worker()	8
6.5	Shared Listen Loop – _listen_loop()	8
6.6	File Transfer – _receive_file() / send_file()	8
6.7	Cleanup – remove_peer()	9
7	Graphical User Interface (gui.py)	9
7.1	Thread Safety	9
7.2	Latest Interface Revision (v3)	9
7.2.1	Windows DPI-awareness fix	9
7.2.2	Per-peer accent colour	10
7.2.3	Per-peer chat surfaces	10
7.3	Key Event Handlers	11
8	Project Structure	11
9	Installation and Execution	11
9.1	Requirements	11
9.2	Running the Application	12
9.3	Connecting Two Peers	12
10	Testing and Results	12
10.1	Starting a Peer	12
10.2	Two Peers Connected	13
10.3	Text Chat	13
10.4	File Transfer	14

10.5 Error Handling	14
11 Limitations	15
12 Conclusion	16
13 References	16

1 Abstract

“Jogajog” is a peer-to-peer chat and file-sharing application developed in Python. The application follows a decentralized communication model in which each peer operates as both a TCP server and a TCP client, allowing peers to establish direct connections using IP addresses and ports without relying on a central server.

The application supports real-time text messaging and binary file transfer between connected peers. A custom application-layer protocol using length-prefixed JSON messages is implemented over TCP to provide reliable message framing. The networking layer uses multiple threads to handle incoming and outgoing connections concurrently, while a Tkinter-based graphical interface provides separate conversations for connected peers.

The project includes the implementation of the networking layer, communication protocol, graphical interface, peer connection management, file transfer, and error handling. This report describes the system architecture, protocol design, implementation of the major components, project structure, testing results, and known limitations.

2 Introduction

Most everyday applications (web browsers, messaging apps, email) rely on a central server that every client connects to. A peer-to-peer (P2P) architecture removes that central point: each participant, called a peer, is capable of both offering and consuming services directly to and from other peers. This project was built to demonstrate the fundamentals of P2P networking using raw TCP sockets, without relying on any third-party networking library.

The assignment brief for CSE 433 required a working, GUI-based demonstration of direct peer-to-peer communication: peers must be able to discover and connect to one another manually (by IP and port), exchange text messages, and transfer files of arbitrary type and size, while remaining responsive to multiple peers at once. Blockchain-specific concepts such as consensus, mining, or a distributed ledger were explicitly out of scope – the focus is the networking layer that such systems are typically built on top of.

3 Objectives

The main objectives of the project are:

- Implement a peer that is simultaneously a TCP server and a TCP client (“Server + Client” model).
- Design and implement an application-level message protocol on top of raw TCP, solving the message-framing problem.
- Support direct, two-way text chat between any two connected peers.
- Support transfer of arbitrary binary files (images, audio, video, documents, archives) between peers, streamed in fixed-size chunks rather than loaded entirely into memory.
- Support multiple simultaneous peer connections using a multi-threaded design.
- Provide a usable graphical interface (Tkinter) that a non-technical user can operate: start a peer, connect to another peer, chat, and send files.

- Handle common failure cases (invalid input, refused connections, timeouts, unexpected disconnects) gracefully, without crashing.

4 System Architecture

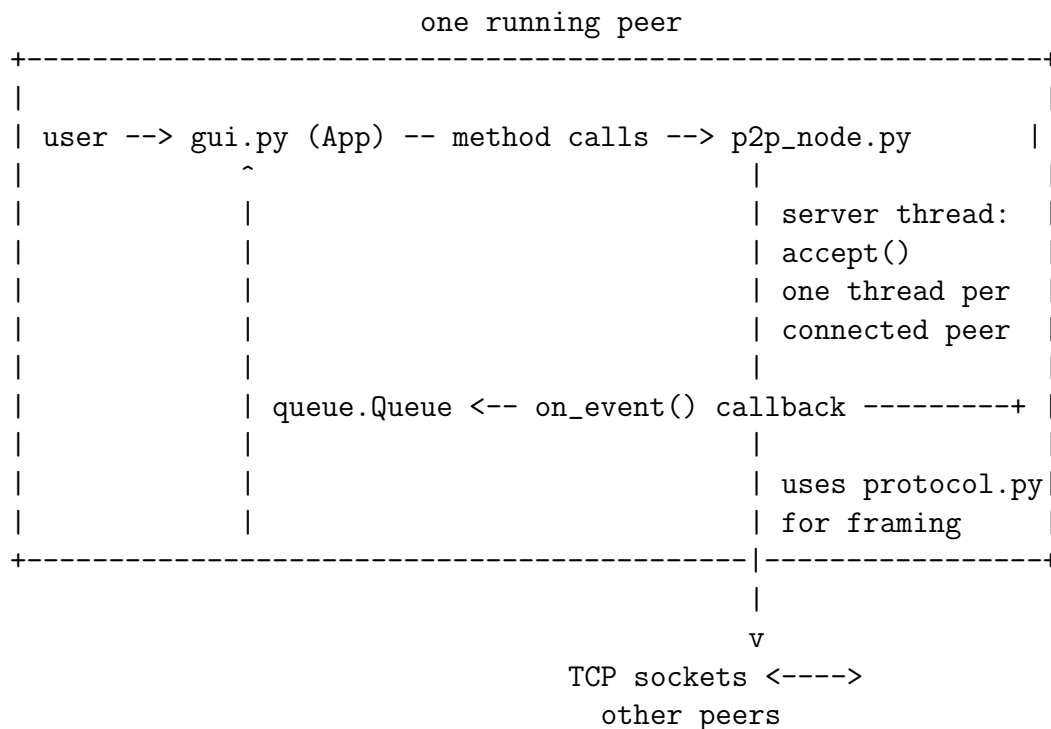
The application is organised into four Python modules, each with a single, clearly separated responsibility. This separation means the networking logic has no dependency on Tkinter and could, in principle, be reused with a different interface or tested headlessly.

Table 1: Main Python modules and their responsibilities

Module	Responsibility
<code>main.py</code>	Entry point. Creates the Tk root window, builds the <code>App</code> , and starts the Tkinter event loop.
<code>gui.py</code>	All user interface code (Tkinter widgets, layout, colors, event handling). Contains no socket code.
<code>p2p_node.py</code>	All networking code: the <code>PeerNode</code> class, which opens a listening socket, accepts incoming connections, opens outgoing connections, performs the handshake, and sends/receives text and files.
<code>protocol.py</code>	Defines the wire protocol: message framing (length prefix + JSON), message type constants, and peer ID generation.

4.1 High-Level Data Flow

The high-level data flow of one running peer can be represented as:



The GUI never touches a socket directly, and the networking layer never touches a Tkinter widget directly. The only channel between them is a small, well-defined interface: the GUI calls plain methods on `PeerNode` (`start`, `stop`, `connect_to_peer`,

`send_text`, `send_file`), and `PeerNode` reports everything that happens back through a single `on_event(event_type, **data)` callback.

4.2 Threading Model

The application uses multiple background threads:

- One background thread runs the server's `accept()` loop for the lifetime of the peer.
- One additional thread is spawned per connected peer (incoming or outgoing) to continuously wait for messages from that peer, so the peer can hold many conversations at once without any one of them blocking the others.
- Each outgoing connection attempt (`connect_to_peer`) also runs in its own short-lived thread, so a slow or unreachable remote address cannot freeze the GUI.
- Because Tkinter is not thread-safe, none of these background threads are allowed to touch a widget. Instead, they push events onto a thread-safe `queue.Queue`, which the GUI drains on its own thread every 100 ms using `root.after()`.

4.3 Handshake Sequence

Before two peers can exchange text or files, they perform a two-way handshake so that each side learns the other's peer ID, display name, and listening port. The listening port is not necessarily the same as the ephemeral port the operating system assigned to the outgoing TCP connection.

Peer B (connecting / "client")	Peer A (listening / "server")
----- TCP connect() ----->	
----- hello {id,name,port} ---->	
<----- hello_ack {id,name,port} --	
both sides now register each other	
and start the shared listen loop	

5 Application-Level Protocol (protocol.py)

TCP guarantees a reliable, ordered stream of bytes – it has no concept of “messages.” If two JSON messages are sent back-to-back, a single `recv()` call on the receiving end might return them glued together, or cut one of them in half. This is the classic TCP message-framing problem.

`protocol.py` solves it by prefixing every message with its own length:

```
[ 4 bytes: length of the JSON payload, big-endian ]  
[ N bytes: the JSON payload ]
```

The receiver always performs two reads: first exactly 4 bytes to learn how many more bytes to expect, then exactly that many bytes, which are then parsed as JSON. This guarantees that message boundaries are never lost, regardless of how the underlying TCP stack happens to split the data into packets.

5.1 Constants

Table 2: Protocol constants

Name	Value / Purpose
LENGTH_PREFIX_SIZE	4 – number of bytes used for the length header (supports messages up to approximately 4 GB).
FILE_CHUNK_SIZE	64×1024 (64 KB) – size of each chunk when streaming file bytes, keeping memory usage low.
MSG_HELLO / MSG_HELLO_ACK	Handshake request and reply.
MSG_TEXT	A chat message.
MSG_FILE	Metadata describing a file transfer that is about to start (name + size); the raw bytes follow separately.

5.2 Key Functions

5.2.1 `generate_peer_id()`

Returns the first 8 hexadecimal characters of a random `uuid4`, for example `a83f21c4`. This is short enough to display comfortably in the peer list and unique enough for a small, classroom-scale network.

5.2.2 `encode_message(message: dict) -> bytes`

Serialises the dictionary to UTF-8 JSON, measures its length, packs that length into 4 bytes with `struct.pack(">I", length)` (big-endian, unsigned 32-bit), and returns the length prefix concatenated with the payload.

5.2.3 `recv_exact(sock, num_bytes) -> bytes | None`

This is the most important function in the protocol. A single `sock.recv(n)` call is only guaranteed to return “up to *n*” bytes – it may return fewer if the data has not fully arrived yet.

`recv_exact` loops, accumulating bytes into a buffer, until either exactly `num_bytes` have been collected (success) or the socket returns an empty read, meaning the peer disconnected mid-message (returns `None`).

5.2.4 `recv_message(sock) -> dict | None`

Reads the 4-byte header with `recv_exact`, unpacks it into an integer with `struct.unpack`, reads that many more bytes, and parses them as JSON – the exact mirror image of `encode_message`.

5.2.5 `send_message(sock, message)`

Encodes a dictionary and sends it with `sock.sendall()`, which, unlike `sock.send()`, loops internally until every byte has actually been handed to the operating system.

5.3 Message Types

Table 3: Application message types

Type	Direction / Purpose	Fields
hello	Sent by the connecting peer immediately after <code>connect()</code>	<code>peer_id</code> , <code>peer_name</code> , <code>port</code>
hello_ack	Reply from the accepting peer, completing the handshake	<code>peer_id</code> , <code>peer_name</code> , <code>port</code>
text	A chat message	<code>sender_id</code> , <code>sender_name</code> , <code>message</code>
file	Metadata sent before raw file bytes	<code>sender_id</code> , <code>sender_name</code> , <code>filename</code> , <code>filesize</code>

6 Core Networking Layer (`p2p_node.py`)

This module implements the `PeerNode` class, which is the heart of the project: every instance is simultaneously a server (so other peers can connect to it) and a client (so it can connect out to others).

6.1 State Held by a `PeerNode`

Each `PeerNode` maintains:

- `name`, `port`, `peer_id` – this peer’s own identity.
- `downloads_dir` – folder where received files are written; created automatically if missing.
- `on_event` – a callback invoked from worker threads whenever something the GUI should know about happens (connected, disconnected, message received, file received, error). Defaults to a no-op so `PeerNode` can be used independently of any GUI.
- `_connections` – a dictionary mapping peer ID to `{socket, name, ip, port}` for every currently connected peer.
- `_connections_lock` – a `threading.Lock` guarding `_connections`, since it is read and written from the accept loop, every per-peer listen thread, and the GUI thread.

6.2 Starting and Stopping – `start()` / `stop()`

`start()` follows the standard socket $\rightarrow$ bind $\rightarrow$ listen $\rightarrow$ accept pattern. It creates an IPv4 TCP socket, sets `SO_REUSEADDR` (so the peer can be restarted on the same port immediately without waiting for the operating system to release it), binds to `("0.0.0.0", port)` so the peer accepts connections on every network interface (not just localhost), and calls `listen()`.

The accept loop is then handed off to a daemon background thread so it never blocks the caller.

`stop()` flips a running flag to `False`, closes the server socket (which causes the blocked `accept()` call to raise `OSError` and exit the loop), and then removes every connected peer

without sending disconnect notifications, since the local user is the one leaving.

6.3 Accepting Connections – `_accept_loop()` / `_handle_incoming_connection()`

`_accept_loop()` blocks on `accept()` until a new TCP connection arrives, then immediately spawns a dedicated thread to run `_handle_incoming_connection()` for that connection and loops back to accept the next one. This is what allows several peers to connect at the same time.

`_handle_incoming_connection()` performs the server side of the handshake: it waits for a hello message, validates its type, replies with `hello_ack` containing this peer's own identity, registers the new peer in `_connections`, and then falls into the shared `_listen_loop()`.

6.4 Connecting Out – `connect_to_peer()` / `_connect_worker()`

`connect_to_peer()` only starts a background thread (`_connect_worker`), so a slow or unreachable address never freezes the caller.

`_connect_worker()` opens a socket with a 5-second connection timeout (to fail fast rather than hang indefinitely), connects, switches the socket back to blocking mode, sends a hello message, and waits for `hello_ack`.

Three distinct failure cases are handled explicitly and reported through `on_event` rather than raised as uncaught exceptions:

- `ConnectionRefusedError`: nothing is listening on that port.
- `socket.timeout` / `TimeoutError`: the address did not respond within 5 seconds.
- Any other `OSError`: for example, an unreachable network.

6.5 Shared Listen Loop – `_listen_loop()`

Once the handshake completes – on either the server or client side – both ends run the same `_listen_loop()`.

It repeatedly calls `protocol.recv_message()` and dispatches based on the message's `type` field:

- A text message is reported directly through `on_event`.
- A file message hands off to `_receive_file()`.
- Any other message type is reported as an error.

When `recv_message()` returns `None` (the peer disconnected), the loop breaks and calls `_remove_peer()`.

6.6 File Transfer – `_receive_file()` / `send_file()`

File transfer is a two-step process. The sender first sends a file metadata message containing the filename and exact byte count; it then opens the file in binary mode and streams it in `FILE_CHUNK_SIZE` (64 KB) chunks with `conn.sendall()`, without ever holding the whole file in memory.

On the receiving side, `_receive_file()` reads the metadata, strips any directory component from the filename with `os.path.basename()` (a deliberate security measure: with-

out this, a malicious filename such as `../..../evil.py` could write outside the `downloads/` folder), and then loops, calling `conn.recv()` for at most the remaining number of bytes until exactly `filesize` bytes have been written to disk.

An empty read before the expected size is reached is treated as a disconnect error rather than a silent truncation.

6.7 Cleanup – `_remove_peer()`

When a peer disconnects (or the local peer stops), `_remove_peer()` removes the entry in `_connections` under the lock and calls `socket.shutdown(SHUT_RDWR)` before `close()`.

The shutdown call matters because the same socket’s own listen thread may be blocked inside `recv()` at that exact moment; a bare `close()` from a different thread does not reliably wake a syscall already in flight on the same file descriptor, whereas `shutdown(SHUT_RDWR)` forces the blocked `recv()` to return immediately with an end-of-file signal and properly notifies the remote peer that the connection has closed.

7 Graphical User Interface (`gui.py`)

The interface is built with Tkinter and is organised as a sidebar (identity, connection controls, and the list of connected peers) next to a chat area (a header showing who is selected, a scrollable message view, and an input bar).

The GUI’s public surface – the `App(root)` class – has stayed the same since the project’s first version, so `main.py` has never needed to change even as the interior of `gui.py` evolved.

7.1 Thread Safety

`PeerNode` calls `on_event(...)` from background networking threads, but Tkinter widgets may only be touched from the main thread.

The GUI’s callback therefore does nothing more than push the event onto a `queue.Queue`. A repeating `root.after(100, self._poll_events)` call drains that queue on the main thread every 100 ms and only then updates any widgets. This is the single mechanism that keeps the whole application thread-safe.

7.2 Latest Interface Revision (v3)

The current version of `gui.py` adds three improvements over the first working version.

7.2.1 Windows DPI-awareness fix

On Windows, Tkinter windows are not DPI-aware by default. If the display’s scaling is set above 100% (common on laptops, e.g. 125% or 150%), Windows renders the window at 100% and then stretches the resulting bitmap to fill the screen, which is what makes text and colors look soft or blurry.

The fix calls `ctypes.windll.shcore.SetProcessDpiAwareness(1)` before any Tk window is created (at import time, in `gui.py`, since `gui.py` is imported before `tk.Tk()` runs in `main.py`), telling Windows to let the application manage its own scaling so Tkinter draws at full, sharp resolution.

The call is wrapped in a `try/except` and is silently skipped on macOS, Linux, and older Windows versions without the `shcore` API.

7.2.2 Per-peer accent colour

Running two instances side by side to simulate two peers used to look identical, since both used the same indigo accent colour.

A small fixed palette of eight (accent, darker-accent, name) tuples is now indexed with `port % len(palette)`, so a peer started on port 5000 always re-themes to the same colour, while a different port will usually land on a different one.

Once `_start_peer()` succeeds, `_apply_accent(port)` re-colours the title, status label, buttons, input-box focus borders, the selected-row colour in the peer list, and the “sent” message colour in the chat view.

A best-effort companion, `_set_titlebar_color()`, goes one step further on Windows 11 and recolours the native OS title bar itself via the Desktop Window Manager API (`DwmSetWindowAttribute`), since Tkinter has no cross-platform way to do this. On any other platform the call is a safe no-op and only the in-window styling changes.

7.2.3 Per-peer chat surfaces

Earlier versions wrote every event – messages to and from every peer, plus all system and error lines – into one shared log.

The current version gives each connected peer its own independent message history, stored in `self.histories` as a dictionary keyed by `peer_id`, with the reserved key `"_system"` used for messages that are not about any one peer, such as “Peer started” or a handshake failure before the remote peer’s identity is even known.

`_record(key, tag, text)` appends a line to the relevant history and only draws it into the visible `Text` widget if that peer’s conversation is the one currently open.

Selecting a different peer in the list calls `_update_chat_title()`, which repoints `current_key` and calls `_render_current_chat()` to wipe and replay that peer’s own history.

The practical effect is that switching between peers now behaves like switching between conversations in an ordinary messenger app, rather than scrolling through one combined feed.

7.3 Key Event Handlers

Table 4: Key GUI event handlers

Handler	Behaviour
<code>_start_peer()</code>	Validates the name and port fields, constructs a <code>PeerNode</code> whose <code>on_event</code> only pushes to the queue, calls <code>start()</code> , and re-themes the UI via <code>_apply_accent()</code> . Reports a clear message if the port is already in use.
<code>_stop_peer()</code>	Stops the node, clears the connected-peers list, and returns the controls to their initial state. The accent colour is deliberately left as-is so the visible history does not jump back to the default colour underneath the user.
<code>_connect_to_peer()</code>	Validates the IP and port fields and calls <code>node.connect_to_peer()</code> .
<code>_send_text()</code>	Requires a selected peer and non-empty text; sends the message and echoes it into that peer's own chat history.
<code>_choose_and_send_file()</code>	Opens the native file picker and calls <code>node.send_file()</code> with the chosen path.
<code>_handle_event()</code>	The single dispatch point for every event type (<code>started</code> , <code>peer_connected</code> , <code>peer_disconnected</code> , <code>text</code> , <code>file_received</code> , <code>file_sent</code> , <code>error</code> , <code>stopped</code>) coming off the queue; routes each one to the correct peer's history (or to <code>"_system"</code>) and updates the corresponding widgets.

8 Project Structure

The project is organised as follows:

```
P2P_Network/  
|-- main.py           # Entry point: creates the window and starts the GUI  
|-- gui.py            # Tkinter user interface (no networking code)  
|-- p2p_node.py        # PeerNode class: sockets, threads, handshake, files  
|-- protocol.py        # Message framing (length prefix + JSON) and constants  
|-- downloads/         # Received files are saved here  
'-- README.md
```

9 Installation and Execution

9.1 Requirements

- Python 3.10 or later (the code uses the `X | None` type-hint syntax).
- No third-party packages are required – only `socket`, `threading`, `json`, `struct`, `uuid`, `queue`, and `tkinter` from the standard library.
- On some Linux distributions, Tkinter must be installed separately:

```
sudo apt install python3-tk
```

9.2 Running the Application

Run the following command:

```
python main.py
```

Running this command more than once (in separate terminals, or on separate computers on the same network) simulates multiple peers.

9.3 Connecting Two Peers

1. Start Peer A: enter a name (e.g. “Alice”) and a port (e.g. 5000), then click **Start Peer**.
2. Start Peer B in a second instance: name “Bob”, use a different port (e.g. 5001), then click **Start Peer**.
3. On Peer B, under **Connect to a peer**, enter Peer A’s IP and port (127.0.0.1 and 5000 on the same machine) and click **Connect**.
4. Both peers now list each other under **Connected Peers**, each tinted in its own accent colour.

10 Testing and Results

The application was exercised manually by running two or more instances on the same machine (using 127.0.0.1) and, separately, on two machines on the same Wi-Fi network.

The scenarios below were verified; screenshots are left as placeholders for the actual run performed for submission.

10.1 Starting a Peer

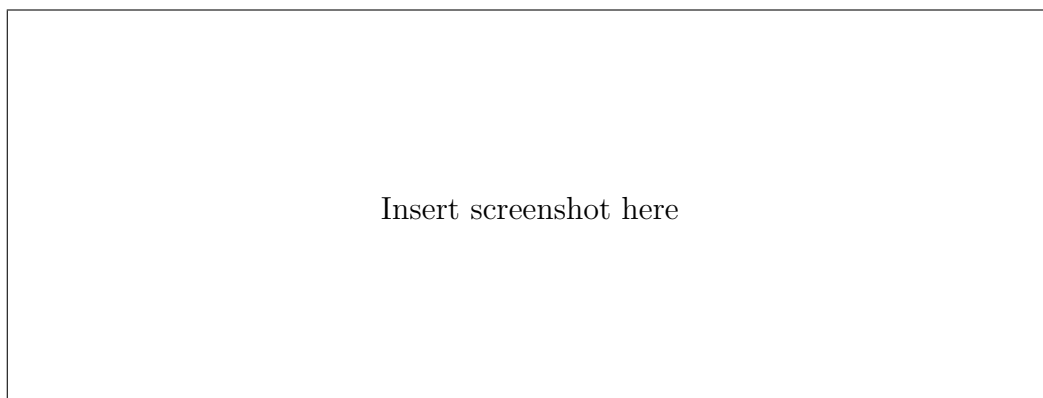


Figure 1: Application window immediately after starting a peer (status shows “Online”, accent colour matches the chosen port).

10.2 Two Peers Connected

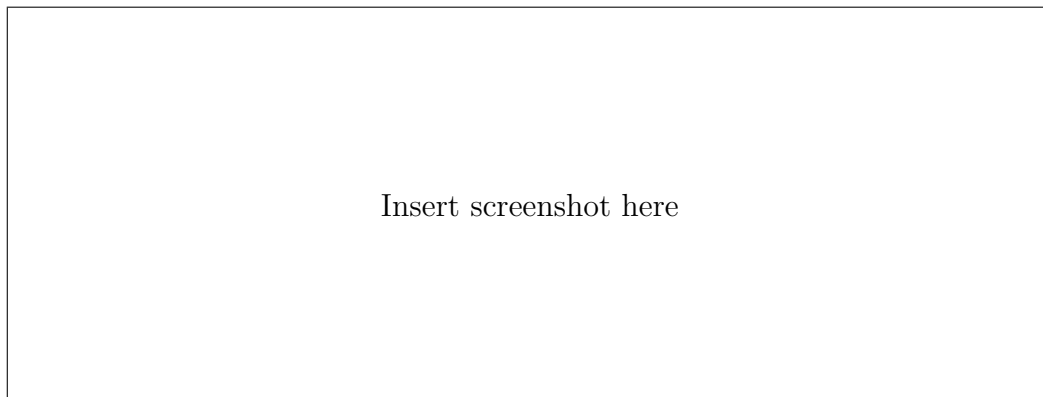


Figure 2: Peer A's window, with Peer B visible in the Connected Peers list.

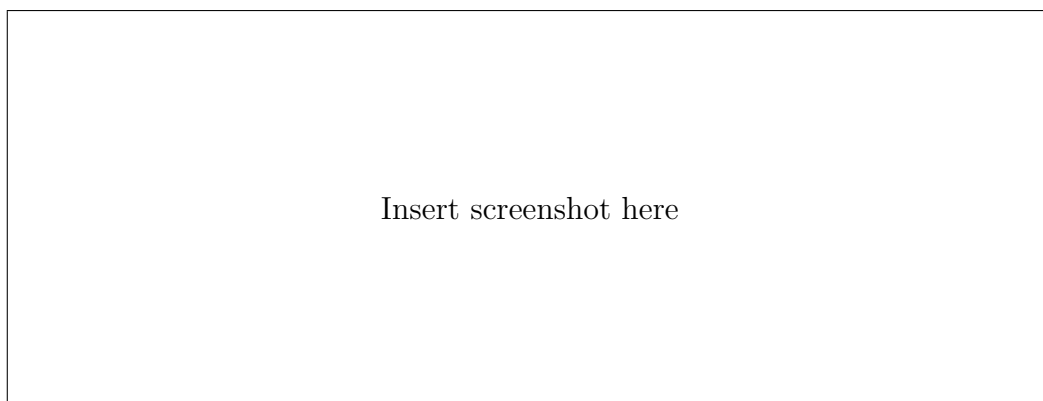


Figure 3: Peer B's window, with Peer A visible in the Connected Peers list, in a different accent colour.

10.3 Text Chat

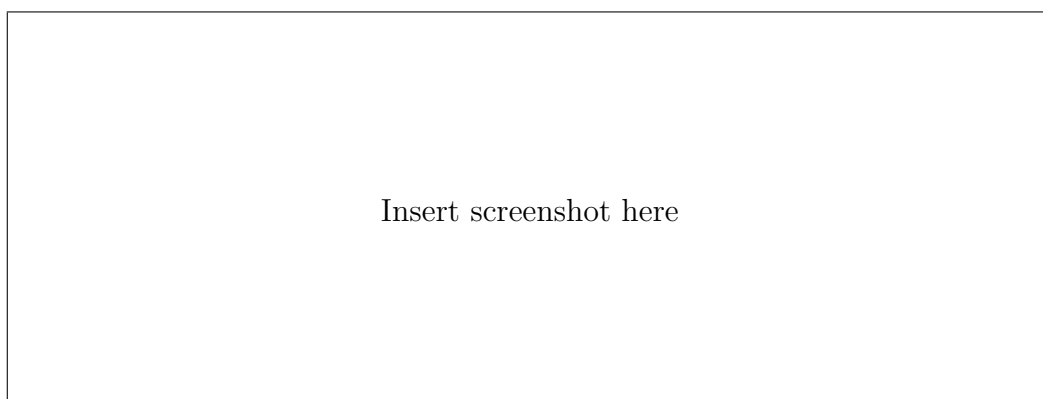


Figure 4: A short exchange of text messages between the two peers, each in its own conversation view.

10.4 File Transfer

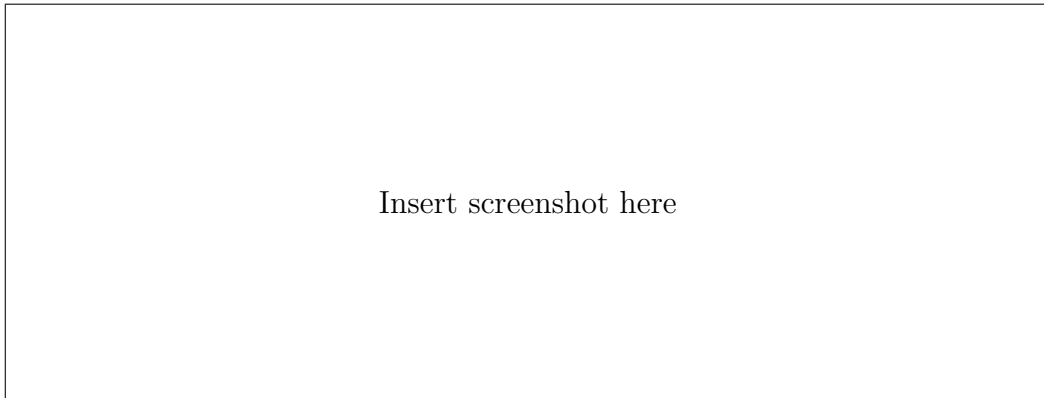


Figure 5: Sender's log confirming a file was sent.

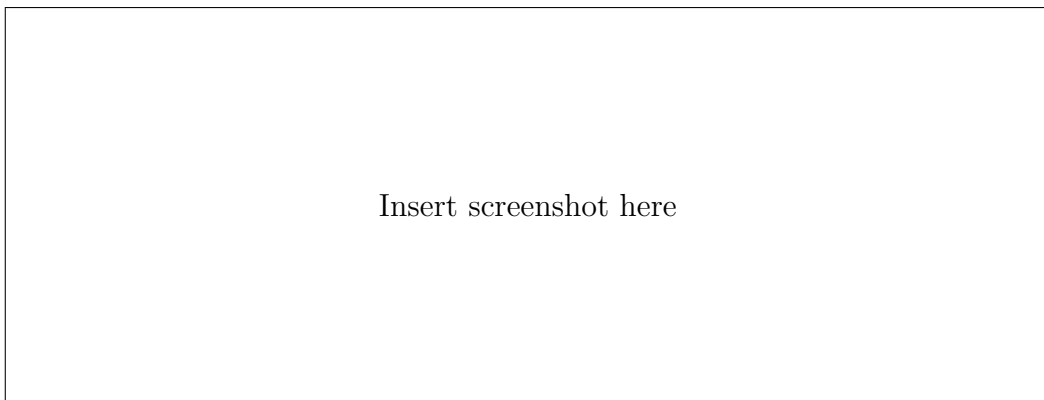


Figure 6: Receiver's log confirming the file was received and saved to `downloads/`.

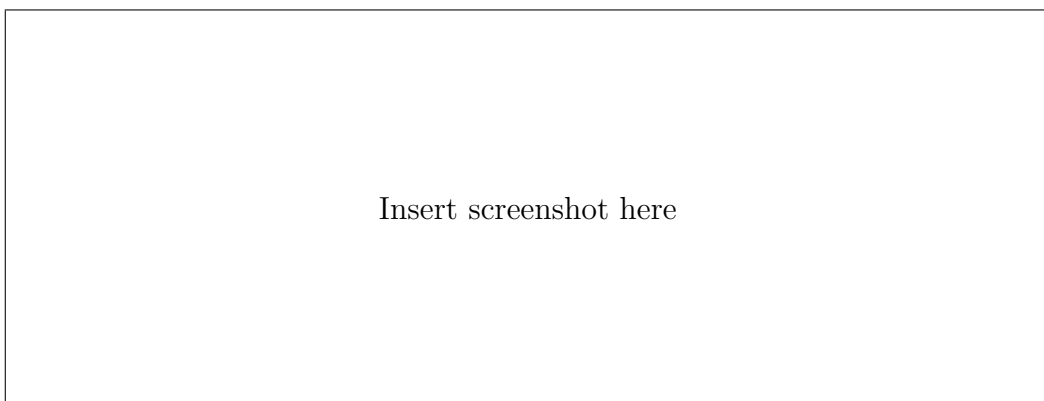


Figure 7: The `downloads/` folder showing the received file.

10.5 Error Handling

The following failure cases were triggered deliberately and confirmed to be reported in the chat log rather than crashing the application.

Table 5: Error-handling test cases

Scenario	Observed Behaviour
Starting a peer on a port already in use	A message box reports “Port may already be in use”; the peer is not created.
Connecting to an IP/port with nothing listening	“Connection refused by <code>ip:port</code> ” is shown in the log.
Connecting to an unreachable address	After a 5-second timeout, “Connection to <code>ip:port</code> timed out” is shown.
Remote peer closes the app mid-conversation	The peer is removed from the Connected Peers list and “ <code>name</code> disconnected” is logged, without affecting other connections.
Non-numeric or out-of-range port entered	An input-validation dialog blocks the action before any socket is opened.

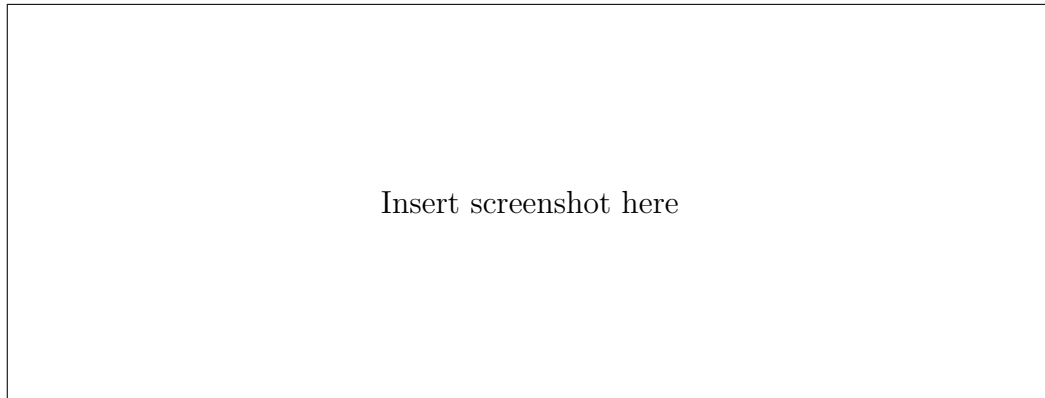


Figure 8: Example error message shown after a refused connection.

11 Limitations

The current implementation has the following limitations:

- **No encryption or authentication** – all messages (including file metadata) travel as plain-text JSON; this was explicitly out of scope for the assignment.
- **No peer discovery mechanism** – peers must already know each other’s IP address and listening port.
- **Filename collision** – a received file with the same name as an existing file in `downloads/` silently overwrites it.
- **GUI blocking during file sending** – `send_file()` runs on the GUI thread, so sending a very large file can briefly freeze the window.
- **Malformed message handling** – if a malformed (non-JSON or unexpected) message is somehow received, `_listen_loop` currently has no catch for `json.JSONDecodeError` specifically inside the loop, which would end that peer’s listening thread.
- **No NAT traversal** – the application works on a local network or between machines with direct IP reachability, not across arbitrary internet connections.

12 Conclusion

This project successfully demonstrates the core ideas behind peer-to-peer networking using nothing more than the Python standard library: every peer plays both the server and client role, a custom length-prefixed protocol solves the TCP framing problem cleanly, and a multi-threaded design lets a single peer hold several simultaneous conversations without blocking.

Layering a thread-safe event queue between the networking code and a Tkinter GUI kept the two halves of the project fully decoupled, which made the interface easy to redesign (twice) without touching a single line of networking code.

The main limitations – no encryption, no peer discovery, and no NAT traversal – were explicitly excluded by the assignment’s scope and would be natural next steps for extending this project toward a production-grade P2P system.

13 References

- Python Software Foundation, “socket – Low-level networking interface,” Python 3 documentation.
- Python Software Foundation, “threading – Thread-based parallelism,” Python 3 documentation.
- Python Software Foundation, “struct – Interpret bytes as packed binary data,” Python 3 documentation.
- Python Software Foundation, “tkinter – Python interface to Tcl/Tk,” Python 3 documentation.
- Course materials, CSE 433 – Blockchain & Distributed Security Lab, assignment specification.